DISCIPLINA DE QUALIDADE DE SOFTWARE

Refatoração de Testes no Projeto Dask

Equipe: Davi Sousa Facundo & José Mykael Alves Nogueira

Critérios de Avaliação

- **40 ocorrências:** Mínimo atingido em dupla.
- **Diversidade:** 6 tipos diferentes de smells resolvidos.
- **Integridade:** Refatorações sem quebrar testes existentes.
- **Contribuição:** Pull Request #12693 submetida.
- **Métricas:** Arquivos estruturados e relatório completo.
- **Complementares:** Formulários e vídeo entregues.

CONTEXTO DO SISTEMA

O Projeto Dask

Python

Computação paralela e distribuída.

16.7k

Testes baseados no Pytest.

65k

Linhas de código de teste.

As 40 Ocorrências Selecionadas

Conditional Logic

Remoção de if/else e loops nos testes.

Exception Handling

Substituição de try/except por asserções nativas.

Verbose Test

Extração de setups complexos para fixtures.

Sleepy Test

Eliminação de pausas arbitrárias do SO.

Non-Functional

Exclusão de código morto e blocos pass.

Redundant Print

Conversão de logs em asserções formais.

Impacto Direto nas Métricas

-46

Ocorrências de Smells

Redução total detectada pelo TEMPY (de 1.783 para 1.737).

+229

Testes Aprovados

Desmembramento de métodos monolíticos (total: 16.951).

84,8%

Cobertura Mantida

Comportamento e caminhos originais preservados integralmente.

REFATORAÇÃO R10

Conditional Test Logic

✕ Antes

```
if io == "ones":
  darr = da.ones(shape, chunks)
elif io == "zeros":
  darr = da.zeros(shape, chunks)
elif io == "full":
  darr = da.full(shape, 10, chunks)
```

Fluxo controlado por estruturas if/elif internas.

✓ Depois

```
# Via @pytest.mark.parametrize

darr = getattr(da, io)(
  shape, *extra_args, chunks
)
```

Despacho dinâmico via parametrização rica.

REFATORAÇÃO R33

Exception Handling

✕ Antes

```
try:
  get({"x": (f,)}, "x")
  assert False
except MyException as e:
  assert "a" in dir(e)
  assert e.a == 1
```

Controle manual mascarando possíveis falhas.

✓ Depois

```
with pytest.raises(
  MyException, match="Erro"
) as exc_info:
  get({"x": (f,)}, "x")

assert exc_info.value.a == 1
```

Validação nativa de atributos com gerenciador de contexto.

REFATORAÇÃO R03

Verbose Test

✕ Antes

```
# 76 linhas de teste
vals = np.array([1348 ... ,  ... ])
breaks = [10, 36, 58]
for i in range(len(breaks)):
   ...  # particionamento manual
assert ddf.is_monotonic
```

Excesso de infraestrutura ocultando o objetivo visual.

✓ Depois

```
# 4 linhas de teste
ddf = _make_unsorted_timestamp_ddf()

assert ddf.set_index( ... ).is_monotonic
```

Extração da complexidade para método auxiliar.

Uso do Catálogo CATSPY

1

Triagem

Priorização por nível de severidade e impacto no código.

2

Planejamento

Consulta aos padrões de solução antes da modificação.

3

Adaptação

Resolução de restrições de processos distribuídos (pickle).

Divisão do Trabalho



Davi S. Facundo

20 Ocorrências Refatoradas

Ocorrências mapeadas: R01 até R20.



José Mykael A.

20 Ocorrências Refatoradas

Ocorrências mapeadas: R21 até R40.



Pull Request Submetida

#12693

794 inserções e 715 remoções submetidas ao repositório oficial.
Nenhuma alteração em código de produção, abrangendo 34 arquivos
de teste no upstream.

Obrigado.

Alguma pergunta sobre as refatorações?